

Tutorial_2_Refactoring_Large_Codebase

Tutorial 2: Refactoring a Large Codebase

Duration: 30-45 minutes **Level:** Intermediate **Prerequisites:** Existing codebase, HelixCode installed

Overview

Learn to use HelixCode's advanced features to refactor legacy code: - Codebase mapping with Tree-sitter - Multi-file atomic edits - Checkpoint snapshots for safety - Context compression for large projects

Step 1: Map the Codebase

```
helixcode map --path /path/to/legacy-project --languages  
go,typescript
```

```
# Output:  
# 📊 Codebase Map Generated  
# Files: 247  
# Functions: 1,834  
# Classes: 156  
# Interfaces: 89  
# Total LOC: 45,231
```

Step 2: Analyze Architecture

```
helixcode analyze --full-context --model gemini-2.5-pro
```

```
# Prompt: "Analyze this codebase and identify:  
# 1. Architectural patterns  
# 2. Code smells  
# 3. Refactoring opportunities  
# 4. Security issues"
```

Step 3: Create Safety Snapshot

```
helixcode snapshot create "Before refactoring auth module"
```

```
# Snapshot created: snap-abc123  
# Commit: 7a3f9d2  
# Files: 247
```

Step 4: Use Plan Mode for Refactoring

```
helixcode plan "Refactor authentication module to:  
- Extract interface for auth provider  
- Add dependency injection  
- Improve error handling  
- Add unit tests  
- Update documentation"
```

Step 5: Multi-File Atomic Edit

```
helixcode edit --transaction \  
  --files "internal/auth/*.go" \  
  --task "Extract AuthProvider interface and implement for JWT"  
  
# Transaction started: tx-xyz789  
# Files to edit: 8  
# Confirm? (y/n): y
```

Step 6: Verify Changes

```
# Run tests  
helixcode test --coverage  
  
# If tests fail, rollback  
helixcode edit rollback tx-xyz789  
  
# Or restore snapshot  
helixcode snapshot restore snap-abc123
```

Step 7: Iterative Refactoring

```
# Use context compression for long session  
helixcode compress --strategy hybrid  
  
# Continue refactoring with smaller context  
helixcode refactor --file internal/user/service.go \  
  --task "Apply same pattern as auth module"
```

Step 8: Auto-Commit Progress

```
helixcode commit --auto  
  
# Generated commit:  
# refactor(auth): extract AuthProvider interface with dependency  
#           injection  
#
```

```
# - Create AuthProvider interface for extensibility
# - Implement JWTAuthProvider with all existing logic
# - Add dependency injection throughout auth module
# - Improve error handling with custom error types
# - Add comprehensive unit tests (95% coverage)
# - Update documentation with architecture diagrams
#
# BREAKING CHANGE: AuthService constructor now requires
    AuthProvider
# Migration guide: See docs/migrations/auth-provider.md
```

Results

- **Before:** 247 files, 45K LOC, monolithic auth
- **After:** 247 files, 44K LOC, modular auth with interfaces
- **Test Coverage:** 62% → 95%
- **Time:** 30 minutes vs. 2-3 days manual

Key Features Used: - Codebase Mapping - Multi-File Edit Transactions - Checkpoint Snapshots
- Context Compression - Auto-Commit

Continue to [Tutorial 3: Using Multiple AI Providers](#)

Sources verified

Sources verified 2026-05-29: <https://go.dev/dl/> (go1.26.3 latest stable Go) ; project go.mod (inner go 1.26) + CLAUDE.md §3.1. **NEGATIVE FINDING** — per CONST-036 the model ID shown (gemini-2.5-pro) **MUST NOT** be treated as a hardcoded authority; LLMsVerifier is the single source of truth for provider/model metadata at runtime. Confirm the exact model IDs available to you with `helixcode llm models list` rather than relying on the example string here.